Global Solution 2026.2

Computational Thinking with Python

Alerta Criadouro

Matheus Rebelo - RM 570260
Felipe Olmos - RM 570946
Luiz Guilherme - RM 573486

1. Problema, público e contribuição da API

Cerca de 75% dos focos do mosquito da dengue ficam dentro das casas (governo de SP, via Poder360, fevereiro de 2026). Em 2024 o Brasil teve 6,48 milhões de casos prováveis e 5.972 mortes, o pior ano da história (Agência Brasil). A larva vira mosquito em cerca de 10 dias, e quase sempre a família só percebe o criadouro quando alguém já ficou doente.

Nosso público são as famílias que moram em casas com quintal em bairros da cidade de São Paulo. Elas recebem o Alerta Criadouro: um aparelho com ESP32 que mede se um potinho sentinela no quintal está com água parada, há quantos dias, e se o clima está quente e úmido.

A API é o lugar para onde o aparelho manda as leituras. Ela confere se a mensagem é válida, classifica de novo cada leitura (OK, ATENÇÃO ou ALERTA), guarda tudo num histórico em CSV e deixa esse histórico disponível para consulta. Assim a família, os agentes de combate a endemias e os modelos de IA das outras matérias conseguem usar esses dados. Sem a API, o aviso ficaria só no LED da área de serviço.

2. Arquitetura e origem dos dados

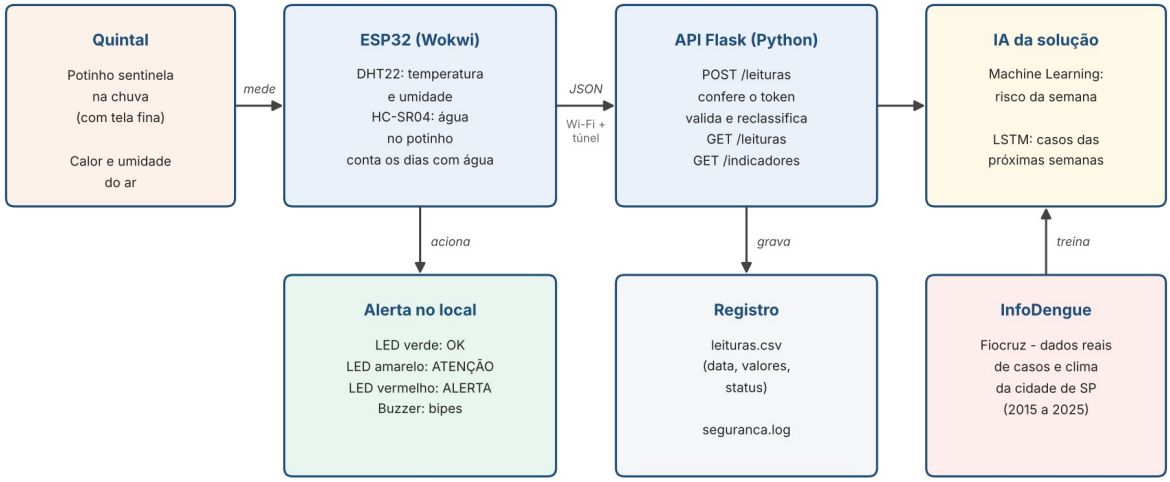


Figura 1 - Arquitetura do Alerta Criadouro

Fluxo: ESP32 com sensores (Wokwi) → JSON → API REST em Flask → processamento (validação e classificação) → leituras.csv → consulta GET.

As leituras vêm dos sensores do ESP32 simulado no Wokwi: DHT22 (temperatura e umidade) e HC-SR04 (distância até a água do potinho). Para testar sem o Wokwi usamos o script testar_api.py, que manda o mesmo JSON. A API roda no Google Colab, e um túnel gratuito da Cloudflare dá um endereço público para o Wokwi alcançar a API pela internet.

3. Contrato JSON

É o mesmo formato usado no ESP32 (AICSS) e na Lambda da AWS. Campos obrigatórios: dispositivo_id, metrica e valor.

```
{
  "dispositivo_id": "esp32-quintal-001",
```

```
  "tipo_dispositivo": "esp32",
  "latitude": -23.541,
  "longitude": -46.471,
  "metrica": "dias_agua_parada",
  "valor": 3,
  "medido_em": "2026-11-06T14:30:00-03:00",
  "leituras": {
    "temperatura_c": 29.4,
    "umidade_pct": 78.0,
    "distancia_cm": 7.5,
    "nivel_agua_cm": 4.5
  },
  "status_sensores": {"temperatura": "ALERTA", "umidade": "ALERTA", "agua": "ALERTA"},
  "status_local": "ALERTA"
}
```

Campo	Tipo	Obrig.	Regra de validação
dispositivo_id	texto	sim	não pode ser vazio
metrica	texto	sim	precisa ser dias_agua_parada
valor	número	sim	dias com água no potinho, de 0 a 365
leituras.temperatura_c	número	não	de -10 a 60 °C
leituras.umidade_pct	número	não	de 0 a 100%
leituras.distancia_cm	número	não	de 0 a 400 cm (alcance do HC-SR04)
latitude / longitude	número	não	só 3 casas decimais (bairro, não a casa)
cabeçalho X-Token-Dispositivo	texto	sim	precisa ser igual ao token guardado no servidor

Respostas da API:

Código	Quando acontece	Corpo
201	leitura aceita e gravada	id, dispositivo_id, valor, status, nivel_risco, alerta
400	corpo não é JSON, falta campo ou valor fora da faixa	erro e a lista de detalhes
401	token ausente ou errado	erro
429	mais de 20 envios por minuto do mesmo IP	erro

4. Rotas da API

Método e rota	O que faz
GET /health	mostra se a API está no ar
POST /leituras	recebe o JSON do ESP32, valida, classifica e grava no CSV
GET /leituras	lista as leituras gravadas; aceita ?dispositivo=... e ?limite=...
GET /indicadores	devolve os números calculados a partir do histórico

5. Como o código foi pensado

Dividimos o problema em partes pequenas, cada uma numa função. Assim fica fácil testar e reaproveitar: a mesma regra de classificação do arquivo regras.py é usada pela API Flask e pela Lambda da AWS.

Conceito	Onde aparece no código
Funções	nivel_agua(), status_dos_sensores() e classificar() em regras.py; validar(), gravar_csv(), ler_csv(), token_valido(), passou_do_limite() e as funções de cada rota em app.py
Condicionais	classificar() decide entre OK, ATENÇÃO e ALERTA; validar() confere cada campo; receber_leitura() devolve 429, 401, 400 ou 201 conforme o caso
Repetições	for nos campos obrigatórios e nas faixas de cada sensor em validar(); for que conta as

	leituras por status em indicadores(); while que limpa os envios com mais de 60 segundos em passou_do_limite()
Coleções	dicionário FAIXAS com os limites de cada sensor; dicionário NIVEL_RISCO; lista COLUNAS_CSV; lista de erros; dicionário de contagem por status; defaultdict com deque para guardar os horários de cada IP

5.1 Regra de classificação

Estado	Regra	nivel_risco
OK	potinho seco (0 dias com água)	normal
ATENÇÃO	tem água, mas não chegou no ponto de alerta	atencao
ALERTA	3 dias ou mais de água com temperatura de 25 °C ou mais e umidade de 70% ou mais, ou 5 dias ou mais em qualquer clima	critico

Por que esses números: a larva vira mosquito em cerca de 10 dias, e mais rápido no calor. Entre 25 e 30 °C o ciclo acelera, e o ar úmido ajuda o mosquito a sobreviver. Avisar no 3° dia (ou no 5°, no frio) dá tempo de a família esvaziar os recipientes.

A API também não confia só no status que o aparelho manda. Ela calcula a classificação de novo com os valores recebidos. Se alguém mandar um JSON falso dizendo OK com água há 6 dias, a API grava ALERTA.

```
def classificar(dias_agua, temperatura=None, umidade=None):
    """Devolve OK, ATENCAO ou ALERTA."""
    if dias_agua <= 0:
        return "OK"
    if dias_agua >= DIAS_ALERTA_SEMPRE:
        return "ALERTA"
    # sem dados de clima, fica só na regra dos dias
    if temperatura is not None and umidade is not None:
        clima_favoravel = temperatura >= TEMP_ALERTA and umidade >= UMIDADE_ALERTA
        if dias_agua >= DIAS_ALERTA_CLIMA and clima_favoravel:
            return "ALERTA"
    return "ATENCAO"
```

6. Persistência em CSV e consulta GET

Cada leitura aceita vira uma linha do arquivo leituras.csv, com a data e hora de São Paulo, os valores e a classificação. Se o arquivo ainda não existe, a API cria e escreve o cabeçalho.

```
data_hora,dispositivo_id,temperatura_c,umidade_pct,distancia_cm,nivel_agua_cm,dias_agua_parada,status,
nivel_risco,alerta
2026-11-05 18:26:07,esp32-quintal-001,22.5,60.0,12.0,0.0,0,OK,normal,False
2026-11-05 18:26:07,esp32-quintal-001,27.0,75.0,8.0,4.0,1,ATENCAO,atencao,False
2026-11-05 18:26:07,esp32-quintal-001,29.4,78.0,7.5,4.5,3,ALERTA,critico,True
```

O GET /leituras lê o CSV e devolve as linhas em JSON. Dá para filtrar por aparelho (?dispositivo=esp32-quintal-001) e limitar a quantidade (?limite=10). Como os dados são de uma casa, a consulta também pede o token.

7. Indicadores calculados

A rota GET /indicadores percorre todo o histórico e calcula o total de leituras e de alertas, quantas leituras ficaram em cada status, o percentual do tempo em alerta, o maior número de dias que o

potinho ficou com água (valor máximo), a temperatura e a umidade médias e qual é o último status do quintal.

Resultado real depois dos três cenários de teste:

```
{"leituras_por_status": {"ALERTA": 1, "ATENCAO": 1, "OK": 1}, "maior_numero_de_dias_com_agua":
3,
 "percentual_em_alerta": 33.3, "temperatura_media": 26.3, "total_alertas": 1, "total_leituras":
3,
 "ultima_leitura_em": "2026-11-05 18:26:07", "ultimo_status": "ALERTA", "umidade_media": 71.0}
```

Para a família, o percentual em alerta e o maior número de dias com água mostram se a rotina de esvaziar os recipientes está funcionando. Para o agente de saúde, o total de alertas ajuda a escolher quais casas visitar primeiro.

8. Testes: três cenários e integração

Os testes foram feitos no dia 05/11/2026 com a API rodando no Google Colab e acessada pela internet, pelo túnel https://vsnet-adequate-spaces-era.trycloudflare.com. O script testar_api.py manda as requisições e mostra as respostas.

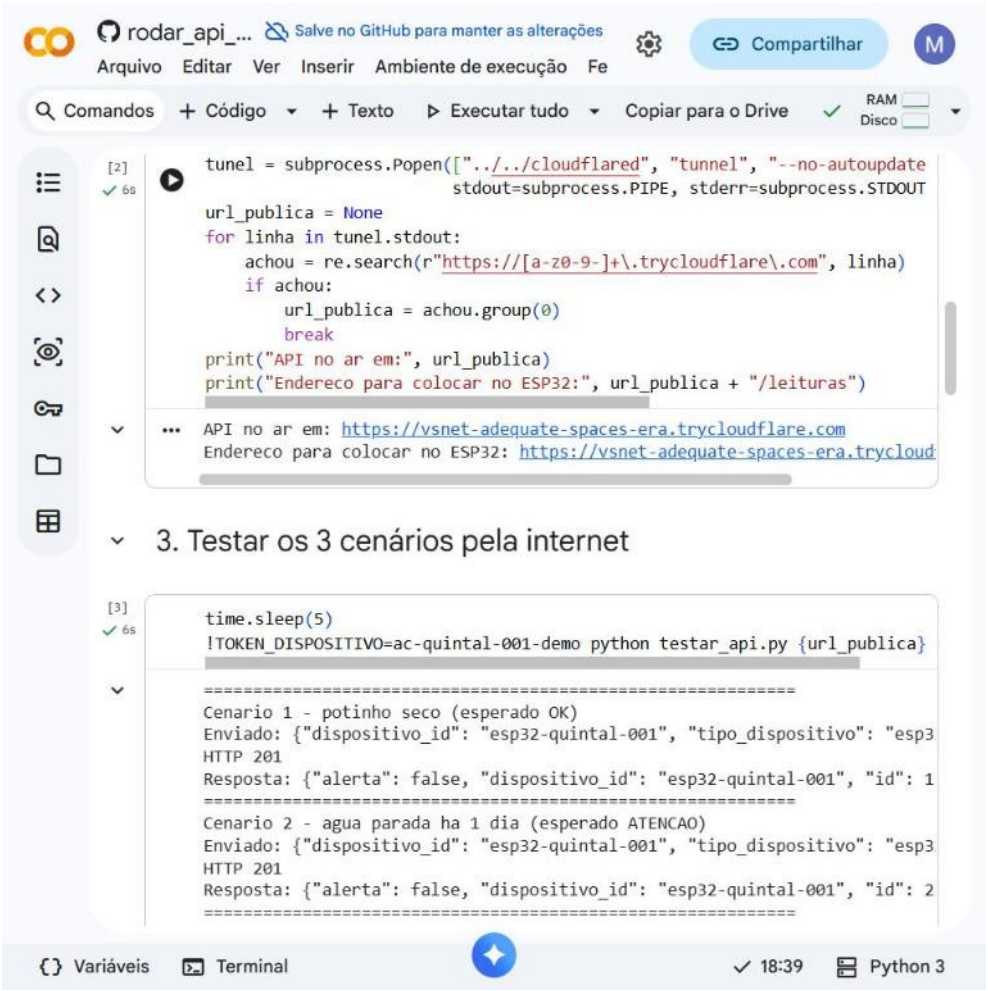


Figura 2 - Execução do testar_api.py no Colab

Cenário 1 - potinho seco (esperado OK)

```
POST /leituras
{"dispositivo_id": "esp32-quintal-001", ..., "valor": 0,
 "leituras": {"temperatura_c": 22.5, "umidade_pct": 60.0, "distancia_cm": 12.0}}
```

```
HTTP 201
{"alerta": false, "id": 1, "nivel_risco": "normal", "status": "OK", "valor": 0}

CSV: 2026-11-05 18:26:07,esp32-quintal-001,22.5,60.0,12.0,0.0,0,OK,normal,False
```

Cenário 2 - água parada há 1 dia (esperado ATENÇÃO)

```
POST /leituras
{"dispositivo_id": "esp32-quintal-001", ..., "valor": 1,
 "leituras": {"temperatura_c": 27.0, "umidade_pct": 75.0, "distancia_cm": 8.0}}

HTTP 201
{"alerta": false, "id": 2, "nivel_risco": "atencao", "status": "ATENCAO", "valor": 1}

CSV: 2026-11-05 18:26:07,esp32-quintal-001,27.0,75.0,8.0,4.0,1,ATENCAO,atencao,False
```

Cenário 3 - água há 3 dias com calor e umidade (esperado ALERTA)

```
POST /leituras
{"dispositivo_id": "esp32-quintal-001", ..., "valor": 3,
 "leituras": {"temperatura_c": 29.4, "umidade_pct": 78.0, "distancia_cm": 7.5}}

HTTP 201
{"alerta": true, "id": 3, "nivel_risco": "critico", "status": "ALERTA", "valor": 3}

CSV: 2026-11-05 18:26:07,esp32-quintal-001,29.4,78.0,7.5,4.5,3,ALERTA,critico,True
```

Testes extras de validação

```
Leitura sem "valor" e com temperatura de 150 °C
HTTP 400
{"detalhes": ["campo obrigatorio ausente: valor", "temperatura_c fora da faixa (-10 a 60)"],
 "erro": "leitura recusada"}

Leitura sem o token do dispositivo
HTTP 401
{"erro": "token do dispositivo ausente ou invalido"}
```

Integração com o ESP32 do Wokwi

Depois dos testes ligamos o ESP32 no Wokwi apontando para a mesma API. O potinho começou seco, depois mudamos a distância do ultrassônico para 7 cm. O histórico mostra a evolução: OK, ATENÇÃO nos dias 1 e 2 e ALERTA a partir do dia 3. Todas as leituras do Wokwi chegaram com HTTP 201.

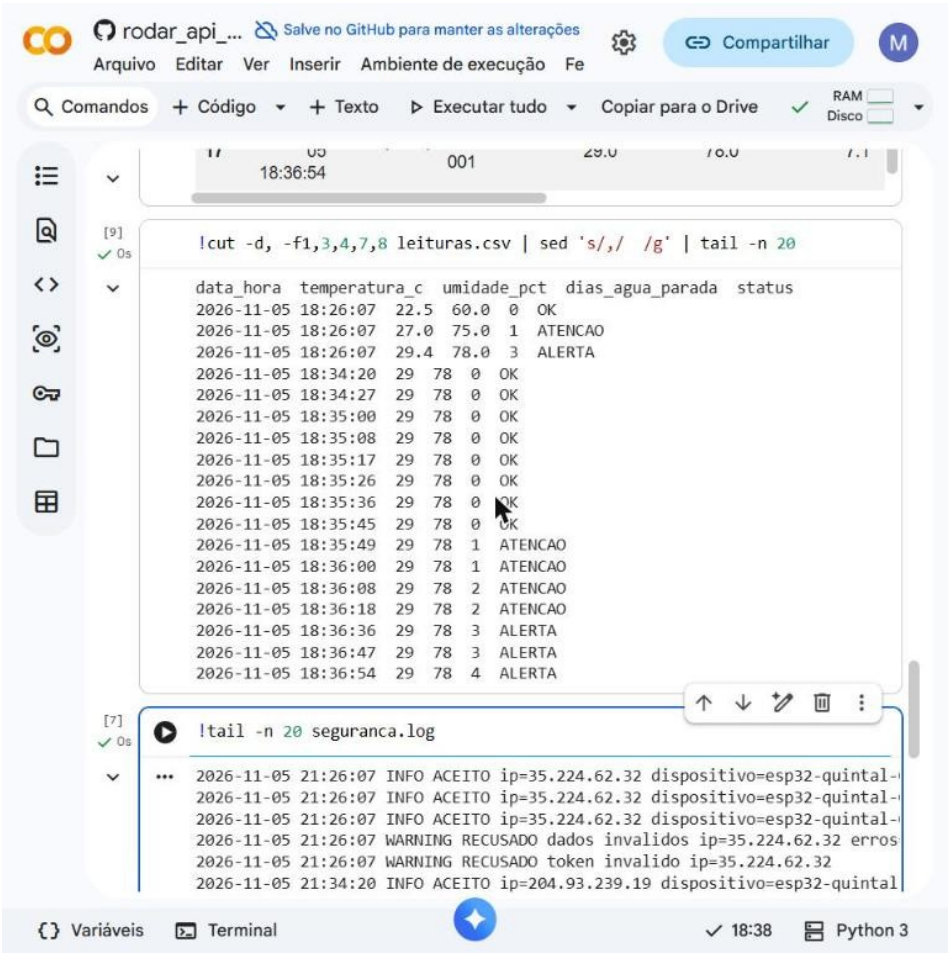


Figura 3 - Histórico CSV com os testes e as leituras vindas do Wokwi

No print, o horário do arquivo seguranca.log aparece em UTC (21:26), 3 horas a mais que o de São Paulo. O CSV já grava no horário de São Paulo (18:26). Depois desse teste ajustamos o código (formato.converter no app.py) para o log também usar o horário de São Paulo.

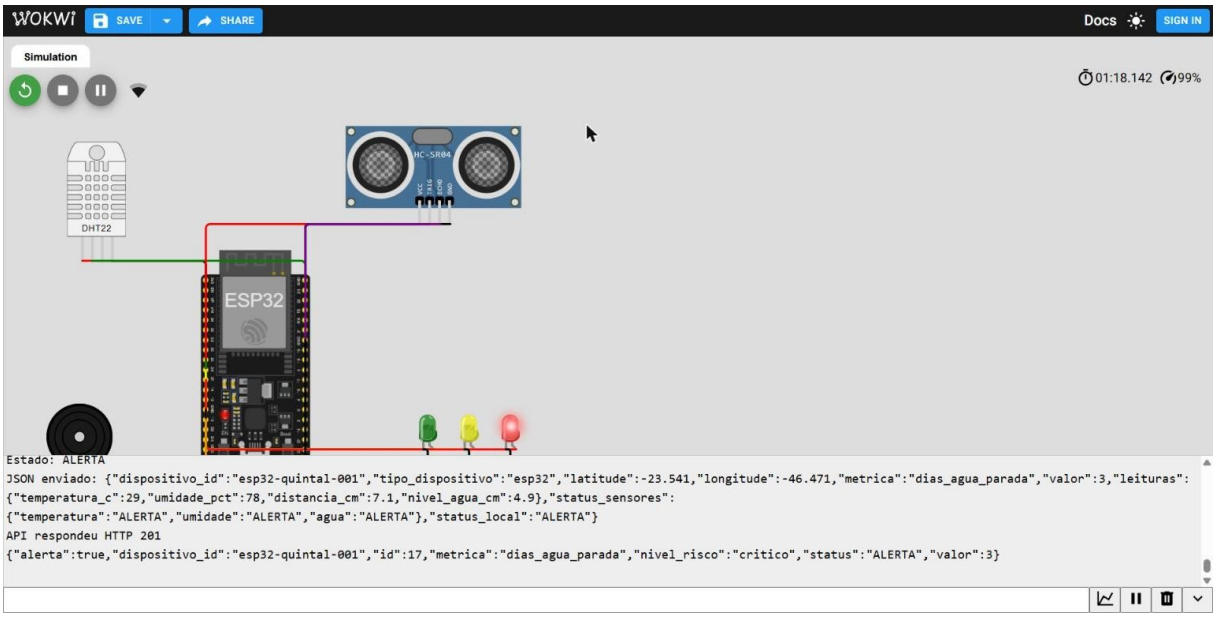


Figura 4 - ESP32 em ALERTA recebendo HTTP 201 da API

9. Como executar

No Google Colab (como fizemos)

1. Abrir https://colab.research.google.com/github/matheusrebelo04-maker/alerta-criadouro/blob/main/api/rodar_api_colab.ipynb
2. Clicar em Ambiente de execução > Executar tudo (e confirmar o aviso do Colab).
3. A célula 2 mostra o endereço público da API. A célula 3 roda os três cenários de teste.
4. As células da parte 4 mostram o CSV e o log gravados.

No computador

```
git clone https://github.com/matheusrebelo04-maker/alerta-criadouro.git
cd alerta-criadouro/api
pip install flask requests

# Linux/Mac
export TOKEN_DISPOSITIVO=ac-quintal-001-demo
python app.py

# Windows (PowerShell)
$env:TOKEN_DISPOSITIVO="ac-quintal-001-demo"
python app.py

# em outro terminal, com o mesmo TOKEN_DISPOSITIVO definido
python testar_api.py http://127.0.0.1:5000
```

10. Limitações

- O CSV é simples e funciona bem para um protótipo, mas com muitas casas mandando dados ao mesmo tempo o ideal seria um banco de dados (na matéria de Cloud usamos PostgreSQL na AWS).
- O limite de requisições fica na memória da API: se ela reiniciar, a contagem zera.
- O token é um só para o aparelho de teste. Na vida real cada aparelho teria o seu próprio token.
- O endereço do túnel do Colab muda toda vez que o notebook é ligado, e o Colab desliga depois de um tempo sem uso.
- No Wokwi o envio foi por http porque a conexão segura estoura o tempo dentro do simulador. Num aparelho real seria https.
- A API classifica cada leitura sozinha. Ela ainda não junta os dados de várias casas para mostrar um mapa do bairro.

11. Relação com os ODS

- ODS 3 - Saúde e bem-estar: o histórico e os indicadores ajudam a família a agir antes de o mosquito nascer, o que reduz casos de dengue.
- ODS 11 - Cidades e comunidades sustentáveis: os dados podem ser usados pelos agentes de saúde e pela prefeitura para planejar visitas no bairro.
- ODS 13 - Ação contra a mudança do clima: a API registra temperatura e umidade, que ligam o clima ao risco de dengue.

Referências

- Poder360. 75% dos focos do mosquito da dengue estão dentro das casas, diz governo de SP. Fevereiro de 2026.
- Agência Brasil. Brasil registra 6,48 milhões de casos prováveis de dengue em 2024.
- Flask. Documentação oficial. https://flask.palletsprojects.com
- Código do projeto: https://github.com/matheusrebelo04-maker/alerta-criadouro

Apêndice A - api/regras.py

```
# Alerta Criadouro - regras de classificação das leituras
# Global Solution 2026.2 - FIAP - 1TIAPZ
# Matheus Rebelo - RM 570260
# Felipe Olmos - RM 570946
# Luiz Guilherme - RM 573486
#
# As mesmas regras do ESP32. Usadas na API Flask e na Lambda da AWS.

ALTURA_POTINHO_CM = 12.0    # distância do sensor até o fundo do potinho vazio
AGUA_MINIMA_CM = 1.0        # a partir disso conta como água parada
TEMP_ALERTA = 25.0          # °C
UMIDADE_ALERTA = 70.0       # %
DIAS_ALERTA_CLIMA = 3       # com calor e umidade altos
DIAS_ALERTA_SEMPRE = 5      # em qualquer clima

# como cada estado aparece na API da AWS
NIVEL_RISCO = {"OK": "normal", "ATENCAO": "atencao", "ALERTA": "critico"}


def nivel_agua(distancia_cm):
    nivel = ALTURA_POTINHO_CM - distancia_cm
    return round(max(nivel, 0.0), 1)


def status_dos_sensores(temperatura, umidade, nivel):
    return {
        "temperatura": "ALERTA" if temperatura >= TEMP_ALERTA else "OK",
        "umidade": "ALERTA" if umidade >= UMIDADE_ALERTA else "OK",
        "agua": "ALERTA" if nivel >= AGUA_MINIMA_CM else "OK",
    }


def classificar(dias_agua, temperatura=None, umidade=None):
    """Devolve OK, ATENCAO ou ALERTA."""
    if dias_agua <= 0:
        return "OK"
    if dias_agua >= DIAS_ALERTA_SEMPRE:
        return "ALERTA"
    # sem dados de clima, fica só na regra dos dias
    if temperatura is not None and umidade is not None:
        clima_favoravel = temperatura >= TEMP_ALERTA and umidade >= UMIDADE_ALERTA
        if dias_agua >= DIAS_ALERTA_CLIMA and clima_favoravel:
            return "ALERTA"
    return "ATENCAO"


if __name__ == "__main__":
    # teste rápido dos três estados
    testes = [
        (0, 30, 80),    # potinho seco
        (1, 30, 80),    # água recente
        (3, 29, 78),    # 3 dias, calor e umidade
        (3, 20, 60),    # 3 dias, clima frio
        (5, 18, 50),    # 5 dias, qualquer clima
        (2, None, None),
    ]
    for dias, temp, umid in testes:
        estado = classificar(dias, temp, umid)
        print(f"dias={dias} temp={temp} umid={umid} -> {estado} ({NIVEL_RISCO[estado]})")
    print("nivel com distancia 7.5 cm:", nivel_agua(7.5), "cm")
    print(status_dos_sensores(29.4, 78, nivel_agua(7.5)))
```

Apêndice B - api/app.py

```
# Alerta Criadouro - API que recebe as leituras do dispositivo
# Global Solution 2026.2 - FIAP - 1TIAPZ
# Matheus Rebelo - RM 570260
# Felipe Olmos - RM 570946
# Luiz Guilherme - RM 573486
#
# Rotas:
#   GET  /health        - mostra se a API está no ar
#   POST /leituras      - recebe o JSON do ESP32, valida, classifica e grava no CSV
#   GET  /leituras      - lista as leituras gravadas
#   GET  /indicadores   - números calculados a partir das leituras
```

```
import csv
import logging
import os
import re
import time
from collections import defaultdict, deque
from datetime import datetime
from zoneinfo import ZoneInfo

from flask import Flask, jsonify, request

from regras import NIVEL_RISCO, classificar, nivel_agua

PASTA = os.path.dirname(os.path.abspath(__file__))
ARQUIVO_CSV = os.path.join(PASTA, "leituras.csv")
ARQUIVO_LOG = os.path.join(PASTA, "seguranca.log")
FUSO = ZoneInfo("America/Sao_Paulo")

# o token não fica no código: vem de variável de ambiente
TOKEN_DISPOSITIVO = os.environ.get("TOKEN_DISPOSITIVO")
LIMITE_POR_MINUTO = int(os.environ.get("LIMITE_POR_MINUTO", "20"))

COLUNAS_CSV = [
    "data_hora", "dispositivo_id", "temperatura_c", "umidade_pct", "distancia_cm",
    "nivel_agua_cm", "dias_agua_parada", "status", "nivel_risco", "alerta",
]

# só letras minúsculas, números e hífen (evita fórmula maliciosa no CSV)
PADRAO_ID = re.compile(r"^[a-z0-9-]{3,40}$")

# faixas aceitas para cada valor (fora disso a leitura é recusada)
FAIXAS = {
    "temperatura_c": (-10, 60),
    "umidade_pct": (0, 100),
    "distancia_cm": (0, 400),
}

app = Flask(__name__)

log = logging.getLogger("alerta_criadouro")
log.setLevel(logging.INFO)
if not log.handlers:
    formato = logging.Formatter("%(asctime)s %(levelname)s %(message)s", "%Y-%m-%d %H:%M:%S")
    formato.converter = lambda *args: datetime.now(FUSO).timetuple()  # horário de São Paulo
    for destino in (logging.FileHandler(ARQUIVO_LOG, encoding="utf-8"), logging.StreamHandler()):
        destino.setFormatter(formato)
        log.addHandler(destino)

# guarda o horário das últimas requisições de cada IP
requisicoes_por_ip = defaultdict(deque)


def ip_do_cliente():
    # atrás do túnel o IP real vem neste cabeçalho
    return request.headers.get("CF-Connecting-IP") or request.remote_addr or "desconhecido"


def passou_do_limite(ip):
    agora = time.time()
    fila = requisicoes_por_ip[ip]
    while fila and agora - fila[0] > 60:
        fila.popleft()
    if len(fila) >= LIMITE_POR_MINUTO:
        return True
    fila.append(agora)
    return False


def validar(dados):
    """Devolve a lista de problemas encontrados no JSON."""
    erros = []
    if not isinstance(dados, dict):
        return ["o corpo precisa ser um objeto JSON"]

    for campo in ("dispositivo_id", "metrica", "valor"):
        if dados.get(campo) is None:
            erros.append(f"campo obrigatorio ausente: {campo}")

    dispositivo = dados.get("dispositivo_id")
    if dispositivo is not None and (not isinstance(dispositivo, str) or not PADRAO_ID.match(dispositivo)):
        erros.append("dispositivo_id invalido (use so letras minusculas, numeros e hifen)")
    if dados.get("metrica") is not None and dados["metrica"] != "dias_agua_parada":
        erros.append("metrica precisa ser dias_agua_parada")

    valor = dados.get("valor")
    if valor is not None:
        if isinstance(valor, bool) or not isinstance(valor, (int, float)):
```

```
            erros.append("valor precisa ser um numero")
        elif not 0 <= valor <= 365:
            erros.append("valor fora da faixa (0 a 365 dias)")

    leituras = dados.get("leituras", {})
    if not isinstance(leituras, dict):
        erros.append("leituras precisa ser um objeto")
        return erros
    for campo, (minimo, maximo) in FAIXAS.items():
        if campo not in leituras:
            continue
        numero = leituras[campo]
        if isinstance(numero, bool) or not isinstance(numero, (int, float)):
            erros.append(f"{campo} precisa ser um numero")
        elif not minimo <= numero <= maximo:
            erros.append(f"{campo} fora da faixa ({minimo} a {maximo})")
    return erros


def gravar_csv(linha):
    arquivo_novo = not os.path.exists(ARQUIVO_CSV)
    with open(ARQUIVO_CSV, "a", newline="", encoding="utf-8") as arquivo:
        escritor = csv.DictWriter(arquivo, fieldnames=COLUNAS_CSV)
        if arquivo_novo:
            escritor.writeheader()
        escritor.writerow(linha)


def ler_csv():
    if not os.path.exists(ARQUIVO_CSV):
        return []
    with open(ARQUIVO_CSV, newline="", encoding="utf-8") as arquivo:
        return list(csv.DictReader(arquivo))


def token_valido():
    token = request.headers.get("X-Token-Dispositivo")
    return bool(TOKEN_DISPOSITIVO) and token == TOKEN_DISPOSITIVO


@app.get("/health")
def health():
    return jsonify({"status": "ok", "servico": "alerta-criadouro"})


@app.post("/leituras")
def receber_leitura():
    ip = ip_do_cliente()

    if passou_do_limite(ip):
        log.warning("BLOQUEADO por excesso de requisicoes ip=%s", ip)
        return jsonify({"erro": "muitas requisicoes, tente de novo em 1 minuto"}), 429

    if not token_valido():
        log.warning("RECUSADO token invalido ip=%s", ip)
        return jsonify({"erro": "token do dispositivo ausente ou invalido"}), 401

    dados = request.get_json(silent=True)
    if dados is None:
        log.warning("RECUSADO corpo nao e JSON ip=%s", ip)
        return jsonify({"erro": "corpo nao e um JSON valido"}), 400

    erros = validar(dados)
    if erros:
        log.warning("RECUSADO dados invalidos ip=%s erros=%s", ip, erros)
        return jsonify({"erro": "leitura recusada", "detalhes": erros}), 400

    leituras = dados.get("leituras", {})
    temperatura = leituras.get("temperatura_c")
    umidade = leituras.get("umidade_pct")
    distancia = leituras.get("distancia_cm")
    nivel = nivel_agua(distancia) if distancia is not None else leituras.get("nivel_agua_cm")
    dias = int(dados["valor"])

    # a API refaz a classificação: não confia só no que o aparelho mandou
    status = classificar(dias, temperatura, umidade)

    linha = {
        "data_hora": datetime.now(FUSO).strftime("%Y-%m-%d %H:%M:%S"),
        "dispositivo_id": dados["dispositivo_id"],
        "temperatura_c": temperatura,
        "umidade_pct": umidade,
        "distancia_cm": distancia,
        "nivel_agua_cm": nivel,
        "dias_agua_parada": dias,
        "status": status,
        "nivel_risco": NIVEL_RISCO[status],
        "alerta": status == "ALERTA",
```

```
    }
    gravar_csv(linha)
    total = len(ler_csv())
    log.info("ACEITO ip=%s dispositivo=%s dias=%s temp=%s umid=%s status=%s",
             ip, dados["dispositivo_id"], dias, temperatura, umidade, status)

    return jsonify({
        "id": total,
        "dispositivo_id": dados["dispositivo_id"],
        "metrica": "dias_agua_parada",
        "valor": dias,
        "status": status,
        "nivel_risco": NIVEL_RISCO[status],
        "alerta": status == "ALERTA",
    }), 201


@app.get("/leituras")
def listar_leituras():
    if not token_valido():
        log.warning("RECUSADO consulta sem token ip=%s", ip_do_cliente())
        return jsonify({"erro": "token ausente ou invalido"}), 401
    linhas = ler_csv()
    dispositivo = request.args.get("dispositivo")
    if dispositivo:
        linhas = [l for l in linhas if l["dispositivo_id"] == dispositivo]
    limite = request.args.get("limite", default=50, type=int)
    return jsonify({"total": len(linhas), "leituras": linhas[-limite:]})


@app.get("/indicadores")
def indicadores():
    if not token_valido():
        log.warning("RECUSADO consulta sem token ip=%s", ip_do_cliente())
        return jsonify({"erro": "token ausente ou invalido"}), 401
    linhas = ler_csv()
    if not linhas:
        return jsonify({"total_leituras": 0, "mensagem": "ainda nao ha leituras"})

    contagem = {"OK": 0, "ATENCAO": 0, "ALERTA": 0}
    for linha in linhas:
        contagem[linha["status"]] += 1

    temperaturas = [float(l["temperatura_c"]) for l in linhas if l["temperatura_c"]]
    umidades = [float(l["umidade_pct"]) for l in linhas if l["umidade_pct"]]
    maior_sequencia = max(int(l["dias_agua_parada"]) for l in linhas)
    total = len(linhas)

    return jsonify({
        "total_leituras": total,
        "total_alertas": contagem["ALERTA"],
        "leituras_por_status": contagem,
        "percentual_em_alerta": round(100 * contagem["ALERTA"] / total, 1),
        "maior_numero_de_dias_com_agua": maior_sequencia,
        "temperatura_media": round(sum(temperaturas) / len(temperaturas), 1) if temperaturas else None,
        "umidade_media": round(sum(umidades) / len(umidades), 1) if umidades else None,
        "ultimo_status": linhas[-1]["status"],
        "ultima_leitura_em": linhas[-1]["data_hora"],
    })


if __name__ == "__main__":
    if not TOKEN_DISPOSITIVO:
        print("Defina a variavel TOKEN_DISPOSITIVO antes de rodar a API")
    app.run(host="0.0.0.0", port=int(os.environ.get("PORTA", "5000")))
```

Apêndice C - api/testar_api.py

```
# Alerta Criadouro - testes da API
# Global Solution 2026.2 - FIAP - 1TIAPZ
# Matheus Rebelo - RM 570260
# Felipe Olmos - RM 570946
# Luiz Guilherme - RM 573486
#
# Manda as mesmas mensagens que o ESP32 manda e mostra o que a API respondeu.
# Uso: python testar_api.py http://127.0.0.1:5000

import json
import os
import sys

import requests
```

```
URL = sys.argv[1].rstrip("/") if len(sys.argv) > 1 else "http://127.0.0.1:5000"
TOKEN = os.environ.get("TOKEN_DISPOSITIVO", "")
CABECALHO = {"X-Token-Dispositivo": TOKEN}


def leitura(dias, temperatura, umidade, distancia):
    return {
        "dispositivo_id": "esp32-quintal-001",
        "tipo_dispositivo": "esp32",
        "latitude": -23.541,
        "longitude": -46.471,
        "metrica": "dias_agua_parada",
        "valor": dias,
        "leituras": {
            "temperatura_c": temperatura,
            "umidade_pct": umidade,
            "distancia_cm": distancia,
        },
    }


def mostrar(titulo, resposta, enviado=None):
    print("=" * 60)
    print(titulo)
    if enviado is not None:
        print("Enviado:", json.dumps(enviado, ensure_ascii=False))
    print("HTTP", resposta.status_code)
    print("Resposta:", json.dumps(resposta.json(), ensure_ascii=False))


cenarios = [
    ("Cenario 1 - potinho seco (esperado OK)", leitura(0, 22.5, 60.0, 12.0)),
    ("Cenario 2 - agua parada ha 1 dia (esperado ATENCAO)", leitura(1, 27.0, 75.0, 8.0)),
    ("Cenario 3 - agua ha 3 dias com calor e umidade (esperado ALERTA)", leitura(3, 29.4, 78.0, 7.5)),
]
for titulo, corpo in cenarios:
    mostrar(titulo, requests.post(f"{URL}/leituras", json=corpo, headers=CABECALHO), corpo)

invalida = leitura(2, 150.0, 78.0, 7.5)
del invalida["valor"]
mostrar("Teste extra - leitura invalida (sem valor e temperatura absurda)",
        requests.post(f"{URL}/leituras", json=invalida, headers=CABECALHO), invalida)

mostrar("Teste extra - sem token do dispositivo",
        requests.post(f"{URL}/leituras", json=leitura(1, 27, 75, 8)))

mostrar("Consulta GET /leituras", requests.get(f"{URL}/leituras", headers=CABECALHO))
mostrar("Consulta GET /indicadores", requests.get(f"{URL}/indicadores", headers=CABECALHO))
```